

TEST DOCUMENTATION

Contents

I. Introduction.....	
1. Project Structure	
2. Introduction	
3. Objectives	
4. Testing Approach	
II. Test Description	
1. Test Object	
2. Tools	
3. Test Plan	
4. Preliminary Test Plan	
5. Supplementary Documents for Executed Test Plan	
6. Final Test Plan	
7. Supplementary Documents for Final Test Plans	
III. Test Project Management	
1. Requirement Coverage	
2. Defect Tracking System and Correction	
3. Defect Reports	
4. Test End Criterion	
5. Final Release Report	
6. Final Product Conformance Confirmation	

I. Introduction

1. Project Structure

The structure of our project:

plumjob/

```
|— src/
|   |— main/
|   |   |— java/...
|   |   └─ resources/
|   └─ test/
|— .env
|— pom.xml
└─ README.md
```

- **main/java** – contains all folders and files responsible for the correct functioning of our application
- **main/resources** – necessary resources
- **test** – folder containing all project test files
- **pom.xml** – file containing all key dependencies and tools used in the project
- **README.md** – file with initial project description

Test structure:

- **unit...** – folders starting with "unit" contain unit tests
- **integration...** – folders starting with "integration" contain integration tests

2. Introduction

PlumJob is an application designed to help young people organize and find a job. The application features include tracking recruitment progress, creating a CV, browsing tips and articles on recruitment, and reminding about upcoming event deadlines.

3. Objectives

We aim to achieve:

- a functional application without critical errors
- an application meeting all functional requirements
- full organization of recruitment processes from start to finish
- seamless CV creation

4. Test Approach

The purpose of testing the project is to verify:

1. that it meets all specified requirements
 2. that all functional requirements work correctly
 3. that the application system is resilient to non-standard user data or behavior
 4. that the application is properly integrated with the database
 5. that there are no implementation errors
 6. that the interface is intuitive and understandable
 7. that the application behaves predictably in all situations
-

II. Test Description

1. Test Object

The test object is the PlumJob application for full management and organization of recruitments, primarily written in Java.

2. Tools

- **Test framework:** JUnit 5 (Jupiter), Spring Boot Test Framework, Mockito
- **Test language:** Java, JavaScript
- **Helper libraries:** Mockito, AssertJ, Spring Security Test, MockMvc
- **Tools:** Testcontainers (Docker Desktop)

3. Test Plan

Testing will be divided into the following stages:

1. **Unit tests** - testing the smallest code units
2. **Integration tests** - testing cooperation between components
3. **Manual tests** - final functionality verification in the running application

Testing responsibility:

- **Tester: Dominik Szymczyk**
- **Team Leader: Kinga Żmuda**

4. Preliminary Test Plan:

Areas to be covered by tests:

1. **Database tests**
 - unit and integration tests
2. **Registration/Login tests**
 - manual and integration tests with the database
3. **Active/Archive Recruitments tests**
 - unit, integration, and manual tests
4. **Articles tests**
 - unit, integration, and manual tests
5. **UserProfile tests**
 - unit and manual tests
6. **Diagram tests**
 - unit, integration, and manual tests
7. **Calendar and Notifications tests**
 - unit and manual tests
8. **CV Builder tests**
 - unit and manual tests

5. Supplementary Documents for Executed Test Plan

Test scheme:

The project distinguishes two main test categories:

- Unit tests
 - Each unit test file is located in `test/java/.../unit\*`
 - Test class is a standard Java class without a public modifier, e.g.
`class ArticleServiceTest { ... }`
 - Tests are written using JUnit 5 and often supported by Mockito (static imports `when()`, `verify()`, etc.)
- Integration tests
 - Located in `test/java/.../integration\*`
 - Classes have the suffix `IT` or end with `IntegrationTest`, e.g. `AdsIntegrationIT.java` or `ArchiveActiveWorkflowIntegrationTest.java`
 - Some integration tests inherit from an abstract base class `IntegrationTestBase`, which starts a MySQL container (Testcontainers) and configures Spring properties

Structure of each test:

- Test class declaration
 - Unit: `class xxxTest { ... }`
 - Integration: `public class XxxIT extends IntegrationTestBase { ... }` when using a container or `XxxIT/XxxIntegrationTest`
- Imports and annotations
 - JUnit 5:
 - `@Test` on test methods
 - `@BeforeEach` (optional) to initialize state before each test
 - `@TempDir` for injecting a temporary file directory
 - Spring (integration only):
 - `@SpringBootTest`
 - `@ActiveProfiles("test")`
 - `@AutoConfigureTestDatabase(replace = Replace.NONE)`
 - Testcontainers (only integration):
 - `@Testcontainers` on base class
 - `@Container` for `MySQLContainer<?> mysql` field
 - `DynamicPropertySource`: to register container URL, username, and password in the Spring context
- Test methods
 - Signature without public, e.g.
 - `@Test`
 - `void readArticle_missingFile_throwsIOException() { ... }`
-

Naming standards:

- Test method names
 - Format:
 - `<method>_<conditions>_<expectedResult>()`
- Test class names
 - Unit: suffix Test, e.g. UserServiceTest.java
 - Integration: suffix IT or IntegrationTest, e.g. AdsIntegrationIT.java or RecruitmentProceduresIntegrationTest.java.
- Packages
 - com.plumdevs.plumjob.unit* for unit tests.
 - com.plumdevs.plumjob.integration* for integration tests.

Test data preparation:

- Resources
 - Helper files in test/resources/*:
 - application-test.properties – database configuration
 - article templates in articles/
 - XML files, images (e.g. avatar.png)
- Data creation
 - Files use ClassPathResource and copying to @TempDir
 - Testcontainers containers automatically create and clean the database
- State management
 - Unit tests recreate a clean service object before each test
 - Integration tests use a separate DB container, isolating state from the local database
- Isolation
 - Each test has its own test class instance (JUnit 5 default)
 - @TempDir ensures a unique file directory
 - DB containers do not share data between suites or the development environment

6. Final Test Plan

Here are only the implemented manual/functional tests conducted on the running application. These tests were executed at the very end, after unit and integration tests (if present). Due to the large number of tests, listing them here would be impractical and make the document too lengthy to read. So the tests that are not here, i.e. unit and integration tests, are in the test folder, respectively.

6.1 Implemented Manual/Functional Tests:

Registration/Login/Logout Tests:

1. Registration with incorrectly formatted email (SUCCESS)
2. Registration with empty fields (SUCCESS)
3. Registration with an already used field (email, username) (SUCCESS)
4. Login of an existing user (SUCCESS)
5. Login of a non-existent user (SUCCESS)
6. Login with incorrect field (username or password) (SUCCESS)
7. Logout of a logged-in user from any view (SUCCESS)

Active/Archive Recruitments Tests:

1. Adding a new entry with correct data (SUCCESS)
2. Adding a new entry with empty fields (SUCCESS)
3. Adding entries with different statuses (SUCCESS)
4. Saving an entry to the database (SUCCESS)
5. Displaying recruitments in Active view (SUCCESS)
6. Filtering by statuses (SUCCESS)
7. Ability to edit statuses (SUCCESS)
8. Inability to edit from final statuses to active (SUCCESS)
9. Full recruitment flow through all possible statuses (SUCCESS)
10. Filtering in Active and Archive by Position/Company/Stage (SUCCESS)
11. Ability to delete a recruitment (SUCCESS)
12. Ability to read recruitment details (SUCCESS)

Diagram Tests:

1. Complete recruitment flow - from addition to diagram (SUCCESS)
2. Multiple recruitments with the same flow (SUCCESS)
3. Different recruitment paths in one diagram (SUCCESS)
4. Adding a new archival recruitment to an existing diagram (SUCCESS)
5. Recruitment with repeating statuses (SUCCESS)
6. Recruitment with status jumps (SUCCESS)
7. Synchronization between Active and Archive (SUCCESS)
8. All possible active → archival transitions (SUCCESS)
9. Long status chain in one recruitment (SUCCESS)
10. Testing jumps available only in Active (SUCCESS)

CV Builder Tests:

1. Loading the basic view (SUCCESS)
2. Navigation to Template 1, 2, 3 (SUCCESS)
3. Adding contact fields (SUCCESS)
4. Removing contact fields (SUCCESS)
5. Finalizing contact fields - empty fields (SUCCESS)
6. Finalizing contact fields - with data (SUCCESS)
7. Finalizing with incomplete number of fields (SUCCESS)
8. Adding education (SUCCESS)
9. Removing education (SUCCESS)
10. Adding/removing skills (SUCCESS)
11. Adding/removing experience (SUCCESS)
12. Adding/removing certificates (SUCCESS)
13. Adding/removing projects (SUCCESS)
14. Adding/removing GDPR clause (SUCCESS)
15. "Finished" button completes editing (SUCCESS)
16. Returning to edit mode (SUCCESS)
17. Generating PDF after Finished (SUCCESS)
18. Required fields validation (SUCCESS)
19. Very long texts in fields (SUCCESS)
20. Special characters in fields (SUCCESS)
21. Maximum number of sections (SUCCESS)

Articles Tests:

1. Does the Tips view contain all articles (SUCCESS)
2. Can each article be read (SUCCESS)
3. Are two articles visible on the right side in the add entry view and can they be accessed and read (SUCCESS)

UserProfile Tests:

1. Loading the "Your Profile" view (SUCCESS)
2. Loading existing profile picture (SUCCESS)
3. Uploading a new picture (SUCCESS)
4. Uploading unsupported format (SUCCESS)
5. Deleting profile picture (SUCCESS)
6. Saving valid preferences (SUCCESS)
7. Attempt to save without selected options (SUCCESS)
8. Loading saved preferences (SUCCESS)
9. Saving a note (SUCCESS)
10. Auto-save to localStorage (SUCCESS)
11. Long note (SUCCESS)
12. Adding a new skill (SUCCESS)
13. Attempt to add duplicate (SUCCESS)
14. Removing a skill (SUCCESS)
15. Extreme level values (SUCCESS)
16. Display match without skills (SUCCESS)
17. Update after adding skills (SUCCESS)

18. Matching for different profiles (SUCCESS)
19. Calculating profile completeness (SUCCESS)
20. Partially completed profile (SUCCESS)
21. Integration with TagService (SUCCESS)

Calendar and Notifications Tests:

1. Adding a new event with all data (SUCCESS)
2. Attempt to add event without selecting date (SUCCESS)
3. Adding event with empty description (SUCCESS)
4. Adding event without time (SUCCESS)
5. Displaying events for the selected date (SUCCESS)
6. Displaying an empty date (SUCCESS)
7. Switching between dates with events (SUCCESS)
8. Deleting an event (SUCCESS)
9. Deleting one of multiple events (SUCCESS)
10. Sending reminder 1 hour before the event (SUCCESS)
11. No duplicate notifications (SUCCESS)
12. Sending multiple notifications for different events (SUCCESS)
13. Handling lack of user email address (SUCCESS)
14. Attempt to add an event in the past (SUCCESS)
15. Adding multiple events on one day (SUCCESS)
16. Handling past and future events (SUCCESS)
17. Correct database save (SUCCESS)
18. Correct sending of notifications at any time (SUCCESS)

Summary:

- Total number of tests (manual/functional): 92
- Tests passed: 92

6.2 Test Coverage of the Application:

- **Code coverage:**
 - Implemented tests covered all parts of the code used in our application. All methods, functions, components, database integration, all functional requirements, every interface, and every view were verified.
- **Functional requirement coverage: 100%**

7. Supplementary Documents for Final Test Plans

Schedule compliance:

- **Planned start date: April 21, 2025**
- **Actual start date: May 1, 2025**
- **Planned end date: June 14, 2025**
- **Actual end date: June 15, 2025**

Additional testing information:

In the functional test plan, it was intended to automate functional tests (e2e). After multiple attempts to write the necessary components and classes for automated testing of our application, unfortunately, it was not fully realized. The problem was that during simulation, the appropriate buttons and fields could not be read, making it impossible to automatically fill in fields and click the appropriate buttons to carry out the tests. For this reason, the application's functional tests were performed manually by the tester. Additionally, I must add that the unit tests for cv builder have been saved in the cv-builder.test.js file and these are tests written exceptionally in JS, because the entire cv builder is written in this language.

III. Test Project Management

1. Requirement Coverage

RC = (number of tested requirements) / (number of requirements recorded in the specification) = 12/12

The only untested requirement by the tester was: "The system should have a counter of people who found a job thanks to it." The reason was that in the code, specifically in the "AboutView", there is a single query to the database where information about the number of people who found a job is stored. The team leader informed the tester that this functionality works and does not need testing.

2. Defect Tracking System and Correction

All defects detected were fixed immediately after testing, whether unit, integration, or manual/functional. Defects found during testing were corrected on an ongoing basis. Each defect found was reported, and then the persons responsible for the respective components were informed of the defects and asked to correct the code immediately.

3. Defect Reports

List of found defects:

1. **Correct sending of notification at any time**
2. **Handling of past and future events**
3. **Attempting to add an event in the past**
4. **Extreme level values**
5. **Attempting to add a duplicate**
6. **Generating PDF after Finished**
7. **The "Finished" button ends editing**
8. **Finalization with an incomplete number of fields**
9. **Finalization of contact fields - with data**
10. **Finalization of contact fields - with empty fields**
11. **Validation of required fields**
12. **Recruitment with a status jump**
13. **Adding a new archived recruitment to an existing diagram**
14. **Impossibility to edit from final to active statuses**
15. **Registration with an incorrectly saved email**
16. **All possible statuses active → archived**
17. **Loading saved preferences**
18. **Saving correct preferences**

4. Test End Criterion

The basic conditions for recognizing the test cycle as completed in our project are:

1. **Business Requirements Verification**
2. **Completeness of Unit and Integration Tests**
3. **Full Passage of Manual Tests**
4. **No Added Value in Subsequent Revisions**

5. Final Release Report

The implemented tests successfully verified key application functions and allowed rapid defect detection. Manual UI tests confirmed full system usability without critical errors. A unified test architecture (DB containers, setup/teardown, data isolation) significantly sped up test writing and maintenance. The entire process ensures high application stability and facilitates future test suite expansion.

6. Final Product Conformance Confirmation

After implementation and testing, the application meets all client requirements. All tests collectively covered over 100% of functionality, minimizing the risk of critical defects. Although a few minor low-priority defects may arise, the tester and team leader ensured stability, performance, and a clear, user-friendly interface. Detailed documentation and consistent use of testing standards guarantee the product is ready for deployment and will satisfy end users.